A New Constructive Lower Bound for The Multiplexer

Marco Carmosino [*], Ngu Dang [†], Tim Jackman [‡]

November 2025

Abstract

Given its nature — selecting a bit from a list of data bits based on an address encoded in binary — the Multiplexing function (MUX) captures an essential form of computation: indirect addressing or memory lookup. Despite its simple nature, proving tight lower and upper bounds for its circuit complexity has remained open for decades. In this work, we revisit the complexity of the multiplexer function MUX and provide two main contributions.

1. We improve the $2N-2$ lower bound (Paul, 1975) to $2N+\log N-2$ via a more intricate application of Gate Elimination (Section 3). Furthermore, we show that this lower bound is also constructive by exhibiting an efficient refuter against circuits of size strictly less than $2N+\log N-2$.
2. We revisit the $2N+O(\sqrt{N})$ upper bound (Klein & Patterson, 1980) and refine the construction by giving exact summation formulas for our recursive constructions and providing a clean and easy to analyze construction that helps provide better understanding on how MUX-function works (Appendix B).

1 Introduction

1.1 Overview

Understanding the circuit complexity of natural Boolean functions is a fundamental pursuit in theoretical computer science for over five decades. Among existing techniques, Gate Elimination — a robust method for proving circuit lower bounds by analyzing the effects of input substitution — has emerged as one of the most tractable and intuitive methods for proving circuit lower bounds. In particular, current known lower bounds in the DeMorgan, $\mathcal{U}_2$ [Red73; Sch74; Pau75; Zwi91; LR01; IM02], and $\mathcal{B}_2$ [Sch74; Sto77; DK11; FGHK16; LY22] basis were proven via Gate Elimination. However, despite its appeal and the long-standing interest, progress in proving strong lower bounds remains limited. To date, no super-linear lower bounds are known for general Boolean circuit classes over different bases, and breaching this barrier via Gate Elimination alone remains elusive [Weg87]. In this work, we consider the DeMorgan Basis: fan-in 2 AND, fan-in 2 OR, and fan-in 1 NOT gates. We consider the complexity measure which counts *only* the binary gates. The table below summarizes the best known upper bound and lower bound of some explicit well-known Boolean functions under our measurement.

Function(s)	Lower Bound		Upper Bound	Source(s)
XOR	$\mathbf{3(n-1)}$	$=$	$\mathbf{3(n-1)}$	[Sch74]
Multiplexer	$2(n-1)$		$2n+O(\sqrt{n})$	[Pau75; KP80]
Sum Mod 4	$4n-O(1)$		$5n-O(1)$	[Zwi91]
Sum Mod 2^k	$4n-O(1)$		$7n-o(n)$	[Zwi91]
Well-Mixed	$5n-o(n)$		poly	[LR01; IM02]
Weighted Sum of Parities	$5n-o(n)$		$5n+o(n)$	[AT11]

Table 1: Explicit Functions with Circuit Lower Bounds in the DeMorgan Basis

[*]IBM, marco@ntime.org
[†]Boston University, ndang@bu.edu
[‡]Boston University, tjackman@bu.edu

In the table, only XOR-function has a proven tight-bound, so attempting to tight the bounds for the other candidates via Gate Elimination (or other known techniques) remains an interesting research direction. In this work, we make progress towards the Multiplexer (MUX) defined below.

Definition. *We define the multiplexer* $\mathsf{MUX}_n : \{0,1\}^n \times \{0,1\}^{2^n} \to \{0,1\}$ *as follows. For all* (a, x) *where* $a \in \{0,1\}^n, x \in \{0,1\}^{2^n}$*, we have*

$$\mathsf{MUX}_n(a, x) = x_{(a)}$$

where $x_{(a)}$ *is the* $((a)+1)$*-th bit of* x *(1-indexed), and the notation* (a) *means the binary string* a *interpreted as a base-2 number.*

For example, if $n = 2$, $a = 01$, and $x = x_1x_2x_3x_4 = 0101$, then $(a) = 1$, thus $\mathsf{MUX}_n(a, x) = x_2 = 1$. Throughout this paper, we will address $a_1, \ldots a_n$ as the *address bits*, and $x_1, \ldots, x_{2^n}$ as the *data bits*, index the function with the number of address bits n, and set $N = 2^n$ as the number of data bits,. We show the following lower bound for MUX_n which an improvement on Paul's $2N - 2$ lower bound [Pau75].

Main Theorem. *(Informal of Theorem 5) Let* C *be an optimal circuit computing* MUX_n*, then under the DeMorgan basis, free negations, the size of* C *is at least* $2N + \log N - 2$ *where* $N = 2^n$*.*

In Appendix B, we provide a detailed analysis the best know upper-bound construction of MUX first introduced by Klein and Patterson [KP80] that is asymptotically close to our lower bound and discuss the feasibility of it being the *optimal* construction. In general, we now obtain

$$2N + \log N - 2 \leq CC(\mathsf{MUX}_n) \leq 2N + O(\sqrt{N}), \text{ where } N = 2^n$$

Furthermore, [CDJ26a] showed that simple lower bound proofs at least in the case of Gate Elimination, like Schnorr's, can be made constructive. Along this line of work, we also show that our $2N + \log N - 3$ lower bound for MUX_n is also constructive (Theorem 6).

1.2 Our Technique

1.2.1 Lower Bound

We first discuss Paul's technique for showing the $2(N-1)$ lower bound. The proof follows a standard gate elimination framework that was used to show the $3(n-1)$ lower bound for XOR_n, the XOR-function on n-bits. In particular, Schnorr analyzed the local structure at the bottom level of an optimal circuit computing XOR_n in which some variable x_i is fed into two different binary gates h, g, and without loss of generality, one can argue h cannot be the output gate which means h is fed into a third binary gate k. Thus, there exists a substitution of x_i such that h, g, k are eliminated. Since restricting x_i yields a circuit computing XOR_{n-1}, i.e. XOR-function is downward self-reducible, one can inductively argue the $3(n-1)$ lower bound.

Paul's lower bound proof follows a similar idea, but the downward self-reducibility of MUX_n behaves differently. Namely, when we restrict an address bit to obtain a circuit computing MUX_{n-1}, we lose half of the current number of data bits. Thus, we cannot apply the downward self-reducibility of MUX-function directly via restricting an address bit using Schnorr's framework. Instead, we will restrict a data bit. However, the resulting circuit after restricting a data bit does not compute MUX_{n-1}, and rather, a function that behaves like MUX_n on every address except one that corresponds to the index of the data bit that was restricted. Thus, we get around this issue by defining a class of function $\mathcal{F} = \{f \mid f$ behaves like MUX on a set of addresses $S \subseteq \{0,1\}^n\}$. Clearly, when $S = \{0,1\}^n$, then $f \equiv \mathsf{MUX}_n$. Thus, if we can obtain a lower bound for the class $\mathcal{F}$, we obtain a lower bound for MUX_n. With this observation, Paul's argument says that at the bottom level of an optimal circuit for f, a restriction on a data bit eliminates at least 2 gates, and the lower bound $2(N-1)$ of MUX_n inductively follows. We recount Paul's argument in Appendix A.

We observe that to obtain a better lower bound for MUX_n, we need to restrict an address bit because the fact that half of the data bits become irrelevant to the circuit can do a lot more damage than just two binary gates at a time. Let C be an optimal circuit computing MUX_n, we first observe the local structure of C around an address bit a_i and the set of $N/2$ data bits that becomes irrelevant to the circuit after substituting $a_i \leftarrow b$. Namely, we argue that a_i and each of the $N/2$ data bits are fed to $N/2+1$ distinct binary gates, as otherwise, we can exhibit an appropriate substitution that makes at least a_i or one of the

data bits degenerate which contradicts the fact that MUX_n depends on all its address and data bits. Thus, at this point, there exists a substitution that eliminates $N/2+1$ binary gates.

Then, we harness Gate Elimination on a "sequence substitution", that is, we take advantage of the fact that if a circuit C has q constants with a cumulative of ℓ fan-outs after substitutions, then we can eliminate at least ℓ more binary gates deeper in the circuit (Lemma 4, a useful gate elimination tool established in [CDJ26b]). Thus, with an appropriate substitution to the $N/2$ data bits, one by one, that were made irrelevant by $a_i \leftarrow b$, we can further eliminate $N/2$ costly gates in a deeper level of the circuit which yields a total of $N+1$ binary gates eliminated at this point.

Finally, restricting $a_i \leftarrow b$ results in a circuit computing MUX_{n-1}, so we can repeat the process above with a different address bit and construct a recurrence that lower bounds the number of gates eliminated after each substitution of an address bit.

1.2.2 Efficient Refuter

To construct an efficient refuter against circuits of size strictly less than $2N + \log N - 3$ that attempts to compute MUX_n, we treat the argument of our lower bound like an "algorithm" and formalize the steps accordingly. In particular, the refuter repeatedly restricts the input circuit along the self-reduction of MUX: at some iteration-i, where $i = n, n-1, \ldots, 2$, it focuses on the half D_i of data bits where the ith-bit of the indices (in binary) of these data bits is 1. Then, for each data bit $x_j \in D_i$, it picks the local out-neighbor gate and assigns x_j a value that fixes this gate to a constant and then simplifies. If x_j ever becomes disconnected or its costly neighbor is the output gate, we can obtain immediately a counterexample because MUX depends on x_j and MUX is sensitive. Otherwise, after repeating the same argument for all x_j in D_i, we can substitute a_i and eliminates at least one more gate because the circuit must still depends on a_i after substituting all data bits in D_i. If the algorithm does not terminate early, then the process continues until only a circuit computing MUX_1 remains which we can brute-force search for a counterexample because the remaining address and data variables are constant.

Our technique for the refuter were inspired by the work of [CDJ26a] where they showed that Schnorr's $3(n-1)$ lower bound for XOR_n is constructive also via treating Schnorr's lower bound as an "algorithm" and the refuter tries to substitute each variable one at a time and builds the counterexample on where the input circuit errs on Schnorr's argument.

1.3 Discussion

An obvious open problem is to further improve our lower bound, as close to the upper-bound as possible. A more ambitious direction is to show that $2N+O(\sqrt{N})$ is a tight bound and that the upper-bound construction is indeed optimal. However, we believe Gate Elimination alone might be too difficult to tight the bound further, let alone characterizing optimal circuits for MUX_n. In Appendix B (Section B.3), we provide some insight on why that is the case and thereby guiding us to investigate other techniques.

2 Preliminaries

2.1 Boolean Circuits

We study general Boolean circuits over the *DeMorgan basis* $\mathcal{B} = \{\wedge, \vee, \neg, 0, 1\}$ of Boolean functions: binary $\wedge$ and $\vee$, unary $\neg$ and zero-ary (constants) 1 and 0. Circuits take zero-ary variables in $X = \{x_1, x_2, \ldots, x_n\}$ for some fixed n as inputs. The standard formulation of circuits consists of single-sink DAGs with nodes labeled by function symbols or variables and edges as "wires" between the gates. We write V_C and E_C to denote the set of nodes and the set of edges of a circuit C respectively, omitting the subscript when it is clear which circuit is being referenced. Boolean circuits compute Boolean functions through *substitution* followed by *evaluation*. Furthermore, we say a circuit C is *well-formed* if it is a fully connected DAG.

An *assignment* of input variables is a mapping from the set of inputs to $\{0, 1\}$. To substitute into a circuit according to an assignment, each input x_i is replaced by the constant. To evaluate a circuit, the values of interior nodes labeled by function symbols are computed in increasing topological order. For each

interior node labeled by a function, it's value is obtained by applying it's function to the value of the node's incoming wires. The output of the circuit overall is the value of its sink.

Let $\mathcal{F}_n$ be the family of Boolean functions on n variables. We say a circuit G on n variables computes $g \in \mathcal{F}_n$ if for all $\alpha \in \{0,1\}^n$, $G(\alpha) = g(\alpha)$. The size of a circuit G, denoted $|G|$, is the number of $\wedge$ and $\vee$ gates in the circuit, and $CC(g)$, the circuit complexity of a Boolean function g, is the minimum size of any circuit computing g. Since only $\wedge$ and $\vee$ gates contribute to circuit size. We say a circuit G computing g is optimal if $|G| = CC(g)$.

2.2 Gate Elimination

Our lower bounds use the classical *gate elimination* method: we fix some inputs to constants, then repeatedly simplify the resulting circuit using local Boolean identities until no further simplifications apply. Intuitively, each simplification either replaces a gate by a constant or merges it into one of its inputs, thereby reducing the number of costly gates while preserving the computed function on the restricted input space. Table 2 lists the local rewrite rules we use, grouped into four types:

- *Fixing* rules: directly turn a gate into the constant 0 or 1 (e.g. $0 \wedge \gamma \to 0$).
- *Passing* rules: replace a gate by one of its inputs (e.g. $1 \wedge \gamma \to \gamma$), letting that input inherit the outgoing wires of the gate.
- *Resolving* rules: eliminate a literal and its negation (e.g. $\gamma \wedge \neg\gamma \to 0$).
- *Pruning* rules: remove duplicate inputs (e.g. $\gamma \wedge \gamma \to \gamma$) and double negations.

Fixing	Passing	Resolving	Pruning
$0 \wedge \gamma \to 0$	$1 \wedge \gamma \to \gamma$	$\gamma \wedge \neg\gamma \to 0$	$\gamma \wedge \gamma \to \gamma$
$\gamma \wedge 0 \to 0$	$\gamma \wedge 1 \to \gamma$	$\neg\gamma \wedge \gamma \to 0$	
$1 \vee \gamma \to 1$	$0 \vee \gamma \to \gamma$	$\gamma \vee \neg\gamma \to 1$	$\gamma \vee \gamma \to \gamma$
$\gamma \vee 1 \to 1$	$\gamma \vee 0 \to \gamma$	$\neg\gamma \vee \gamma \to 1$	
$\neg 0 \to 1$			$\neg\neg\gamma \to \gamma$
$\neg 1 \to 0$			

Table 2: Gate elimination rules

All these rules strictly decrease the number of costly gates and preserve the Boolean function computed by the circuit. We also use a trivial "garbage collection" step: any gate with fan-out 0 that is not the output gate may be deleted. Figure 1 illustrates a passing rule: the child gate γ takes over the outgoing wires of α and α can be deleted once it has fan-out 0.

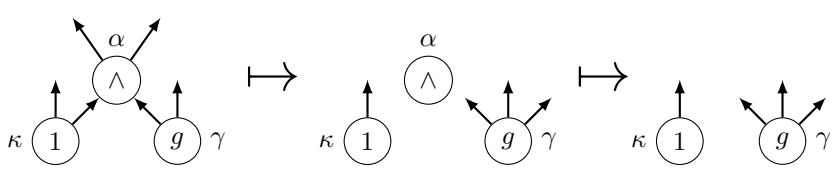

Figure 1: An example of applying the passing simplification $1 \wedge \gamma \to \gamma$. Notice that γ inherits the fanout from α, and α can then be garbage collected.

Note that the identifier of α **never** changes; only its type and incident wires. Depending on the initial fanout of each gate involved in the pattern, applying a rule could totally disconnect some gate(s) and leave the circuit in a state that needs garbage collection. We introduce notation for applying sequences of these steps to circuits, and define composed circuit manipulations that are "well behaved" with respect to specific circuits.

Definition 1. A *simplification* is a sequence of gate elimination and garbage collection steps. If λ is a simplification and C is a circuit, write $\lambda(C)$ to denote the new circuit obtained by applying each step of λ to C in order. If a single step of λ fails to apply in C, then fix $\lambda(C) = C$ so that invalid λ for C are idempotent by convention. A simplification λ is called

- *terminal* for C if, for every λ' that extends λ by one additional gate elimination or garbage collection step, $\lambda(C) = \lambda'(C)$, and/or
- *layered* for C if the depth of **binary** gates binding to α in each step is non-decreasing.

Simplifications formalize every sequence of circuit-manipulation steps except for substitution. They change the function computed by a circuit if and only if some input gate is garbage-collected. Simplifications suffice to impose convenient structural properties on arbitrary circuits.

Definition 2. A circuit C is *normalized* or *in normal form* if

1. C is constant-free **or** C is a single constant,
2. every gate in C has a path to the output, and
3. no sub-circuit of C matches the left hand side of a gate elimination rule in Table 2.

Any terminal simplification suffices to put a circuit C in normal form, and it is straightforward to see that the number of constants in C lower-bounds the number of eliminated gates. We can even push it further: every circuit C can be normalized by a *layered* simplification, and the number of eliminated gates is lower-bounded by the *cumulative fanout* of constants in C (Lemma 4).

Lastly, we emphasize that substitution *always* changes the function computed by C, restricting its domain to a subcube. Additionally, C remains well-formed after any substitution, because input gates have fan-in 0. However, C is *never* in normal form immediately after a substitution because it introduces a constant. So we generalize simplifications by allowing for substitution steps.

Definition 3. A *restriction* is a sequence of substitution, gate elimination, and garbage collection steps. Restrictions generalize simplifications, so we extend notation and conventions accordingly: $\rho(C)$ denotes the result of applying restriction ρ to circuit C, and invalid restrictions for C are idempotent by convention.

The categories of *terminal* and *layered* simplifications apply immediately to restrictions, because they explicitly reference only gate elimination or garbage collection steps. Thus, restrictions are terminal and/or layered only with respect to how they simplify and **not** how they substitute. When applying a restriction ρ to a Boolean function instead of a circuit, we simply ignore the additional simplification steps (if any).

3 Our Improved Constructive Lower Bound

For our lower bound for MUX_n, we take advantage of its downward self-reducibility after substituting one address bit. Such a substitution will make half of the current data bits irrelevant to the resulting circuit computing MUX_{n-1}. It is easy to see that MUX depends on all its address and data bits.

Furthermore, the observation that every circuit C can be normalized by a *layered* simplification, and the number of eliminated gates is lower-bounded by the *cumulative fan-out* of constants in C. This property is immensely useful for our MUX's lower bound. The full proof of this property can be found in [CDJ26b]. Below, we provide a proof sketch.

Lemma 4 ([CDJ26b])**.** *For any well-formed circuit C with q constants that have total fanout ℓ, there is a terminal and layered simplification λ such that $\lambda(C)$ is a single constant* ***or*** $|\lambda(C)| \leq |C| - \ell$.

Proof Sketch. We write ϕ_t as the total fan-out of all constants in C after applying t rule manipulations, and μ_t as the number of binary gates eliminated so far. Note that since a circuit is normalized only when it is a single constant or when $\phi_t = 0$, it is sufficient to exhibit a terminal layered simplification λ such that $\mu_t \geq \ell$ if $\phi_t = 0$. We initialize $\phi_t = \ell$ and $\mu_t = 0$.

Fix an order of the gates in C by depth (breaking ties arbitrarily). Apply gate elimination rules in non-decreasing depth order, interleaving garbage collection of gates with fan-out 0. This yields a layered simplification λ by construction.

We compute the changes to fan-out after each of the 4 types of elimination to show that: (i) each rule application decreases ϕ_t by 1, (ii) whenever ϕ_t decreases by 1, at least one binary gate is eliminated, so μ_t increases by at least 1, and (iii) garbage collection never increases ϕ_t and only increases μ_t. If a single gate remains after λ terminates, then it must be a constant, and we are done. Otherwise, there are no constants with positive fan-out in the simplified circuit, so $\phi_t = 0$. Since each unit dropped in ϕ_t forces an increase of at least 1 in μ_t, and we initialize $\phi_t = \ell$, it follows that $\mu_t \geq \ell$ which yields $|\lambda(C)| \leq |C| - \ell$ as desired. □

Theorem 5. *For the DeMorgan basis $\{\wedge, \vee, \neg\}$ with free $\neg$-gates, we have $CC(\mathsf{MUX}_n) \geq 3$ when $n = 1$ or $CC(\mathsf{MUX}_n) \geq 2N + \log N - 2$ when $n > 1$ and $N = 2^n$.*

Proof. We exploit the downward self-reducibility of MUX_n: substituting an address bit a_i, $i \in [n]$, will eliminate half of the current data bits whose indices are not compatible with the substitution, and the restricted function is a smaller multiplexer. Thus, our strategy is to establish a lower bound on the number of gates eliminated under a specific setting of a_i as well as the data bits that are eliminated, and we repeat this strategy for the remaining address bits, one by one to establish a recurrence on the number of gates eliminated. To this end, let C be an optimal circuit computing MUX_n for $n > 1$, and set $N = 2^n$. Without loss of generality, we focus on a_n, the last address bit. Let D be the set of data bits that becomes degenerate when a_n is set to some binary constant b. It is obvious that $|D| = N/2$, and for convenience and without loss of generality, we index data bits in D as $\{x_j \mid j \in [N/2]\}$. We note that each x_j in D corresponds to a particular setting $a_n = b$ and the remaining data bits outside of D corresponds to $a_n = 1 - b$.

At every stage before x_j is assigned, the current restricted function still depends on x_j. Hence, x_j must feed at least one costly gate g_j. Moreover, g_j cannot be the output gate as otherwise, since the output is a single binary gate, fixing x_j would make the entire circuit constant, a contradiction. Thus, g_j is an internal gate with fanout at least 1.

Without loss of generality, we substitute the variables x_j of D one by one in reverse order with the appropriate fixing rule according to the gate type of its costly successor g_j so that g_j then will become a constant. It is easy to see that the circuit remains well-formed after the substitution. Now apply Lemma 4 to the current circuit after each fixing substitution of x_j. At the moment g_j becomes constant, it has fanout at least 1, so the total constant fanout ℓ increases by at least 1. Lemma 4 therefore yields a layered simplification that deletes at least one additional gate "downstream" of this constant. In particular, the normalized circuit after fixing x_j loses at least 2 gates: one from g_j becomes a constant and at least one more removed by normalization from this constant's fanout. We can repeat this process of eliminating 2 gates with each substitution for the rest of the variables in D because after eliminating some x_j, the circuit must still depends on some $x_k \in D$ at this point. Hence, x_k must be fed into some g_k that is distinct from g_j and similarly, g_k cannot be the output gate. Therefore, eliminating all $|D| = N/2$ variables of D removes at least $2|D| = N$ gates in total at this stage.

After all variables in D are fixed, there remain unfixed data bits outside D (those with addresses correspond to $a_n = 1 - b$). The current restricted circuit must output one of these remaining data bits when setting $a_n = 1 - b$. In other words, the restricted circuit still depends on a_n. Thus, a_n is still fed to a costly gate in the circuit at this point, and substituting a_n will eliminate at least one more costly gate, yielding a total of $N + 1$ gates eliminated at this stage. Furthermore, the simplified circuit computes MUX_{n-1} on $N/2$ data bits by the downward self-reducibility of MUX.

We can further repeat this process for the remaining address bits a_i and the corresponding data bits that they make degenerate and obtain circuit computing smaller multiplexers. In particular, let $CC(\mathsf{MUX}_n) = T(N)$, where $N = 2^n$, which denotes the minimum size of an optimal circuit for MUX_n, then we can lower-bound $T(N)$ with the following recurrence.

$$T(N) \geq \begin{cases} 2, & \text{if } N = 2 \ (n = 1) \\ T(N/2) + N + 1, & \text{if } N > 2 \ (n > 1) \end{cases}$$

The recurring case follows from our analysis, and the base case corresponds to $T(2) = CC(\mathsf{MUX}_1) \geq 3$. Indeed, for this case, we have one address bit (a_1) and two data bits (x_1, x_2), and the circuit must depend on

all of them by the sensitivity of MUX which means at least 2 distinct gates must read all these 3 variables. Furthermore, neither a_1, x_1, nor x_2 can feed the output gate as otherwise, substituting with fixing rules can make the circuit constant which contradicts the sensitivity of MUX. Thus, there exists at least a third costly gate distinct from the previous two which yields the lower bound $CC(\mathsf{MUX}_1) \geq 3$.

We have $n = \log N$ number of address bits, so we can unroll the recurrence as follows

$$\begin{aligned} T(N) &\geq T(N/2) + N + 1 \\ &\geq T(N/4) + N + N/2 + 2 \\ &\geq T(N/8) + N + N/2 + N/4 + 3 \\ &\dots \\ &\geq T(2) + \sum_{i=0}^{\log N - 2} \left(\frac{N}{2^i} + 1 \right) \end{aligned}$$

Note that the recurring case applies for cases $N, N/2, N/4, \dots, 4$ (i.e. $N > 2$) which corresponds to the index of the sum $i = 0, 1, 2, \dots \log N - 2$ respectively. Now, we have

$$\begin{aligned} \sum_{i=0}^{\log N - 2} \frac{N}{2^i} &= N \sum_{i=0}^{n-2} \frac{1}{2^i} \\ &= N \left(\frac{1 - (1/2)^{n-1}}{1 - 1/2} \right) \quad \text{(geometric series for } r = 1/2) \\ &= N(2 - 2^{2-n}) \\ &= 2N - 4 \qquad\qquad (2^{2-n} = 4/2^n = 4/N) \end{aligned}$$

Thus, $CC(\mathsf{MUX}_n) \geq 3 + 2N - 4 + \log N - 1 = 2N + \log N - 2$ as desired. □

We now show that our improved lower bound is also constructive

Theorem 6. *Let C be a DeMorgan circuit on $n + N$ inputs, where $N = 2^n$, with size $s < 2N + \log N - 3$ that claims compute MUX_n. We can construct an input (α, ρ) with $\alpha \in \{0,1\}^n$, $\rho \in \{0,1\}^N$, and $N = 2^n$, such that $C(\alpha, \rho) \neq \mathsf{MUX}_n(\alpha, \rho)$ in polynomial time with respect to N.*

Proof. Let C be a DeMorgan circuit on address inputs $A = \{a_1, \dots, a_n\}$ and data inputs $X = \{x_1, \dots, x_{N-1}\}$, where $N = 2^n$, with $|C| < 2N + \log N - 3$. Algorithm 1 produces a tuple (α, ρ) (for the address and the data bits respectively) and either halts early in the `DEGEN` case or reaches the case `FAIL`, the final constant-size base case. We show that in all cases the algorithm yields a polynomial-time witness that $C \not\equiv \mathsf{MUX}_n$.

For each iteration i, where $i = n, n-1, \dots, 2$, let C_i denote the circuit obtained from C by applying the current partial address assignment α and data assignment ρ and then applying simplification rules. Let $N_i = 2^i$ be the number of data inputs that remain *unfixed* at this point. Define the set

$$D_i \;=\; \{x_j \in X : \text{the } i\text{-th bit of } j - 1 \text{ equals } 1\},$$

so, $|D_i| = N_i/2$. The algorithm maintains the following invariant: if the original circuit C computes MUX_n, then for every $i = n, n-1, \dots, 2$ the simplified restricted circuit C_i computes the corresponding restricted multiplexer, namely MUX_i on the remaining address bits and remaining data bits. Thus, if the input circuit has size $< 2N + \log N$, then it will violate either the invariant at some iteration i of the for-loop (line 6-25) or the final check on MUX_1 (line 27-28).

We first argue that if the algorithm ever reaches the `DEGEN` case (on line 8 or 17), we can efficiently construct an input (α, ρ) witnessing $C(\alpha, \rho) \neq \mathsf{MUX}_n(\alpha, \rho)$. Suppose the algorithm halts in the inner loop because $\text{fanout}(x_j) = \emptyset$ in C_i. Then C_i (hence also C under the current restriction) does not depend on x_j. However, x_j is one of the data bits selected when the i-th address bit is 1, so the restricted multiplexer still depends on x_j. Indeed, fixing the remaining address bits to point to j changes the value of MUX_i when x_j flips. Therefore, under the current restriction we can choose two extensions that differ only on x_j and obtain that C produces the same output on both while MUX_n produces different outputs. This gives a falsifying

assignment. Similarly, if the algorithm ever reaches the `CONST` case (on line 12 or 21), then we can take any completion of ρ and α to construct a valid counterexample.

The same reasoning applies if the algorithm halts because $\text{fanout}(a_i) = \emptyset$, then C_i does not depend on address bit a_i, but the restricted multiplexer MUX_i still depends on a_i (since varying a_i swaps which half of the remaining data bits is being addressed). Again we can extend the current restriction so that MUX_n changes but C does not, yielding a falsifying assignment.

Now we assume that the algorithm does not reach the `DEGEN`. We argue that each iteration i enforces the deletion of at least $N_i + 1$ gates from the current circuit. For each $x_j \in D_i$, we have $\text{fanout}(x_j) \neq \emptyset$, so the algorithm picks a gate g_j adjacent to x_j and assigns x_j to the value $\mathsf{fixer}(x_j, g_j)$ that triggers a fixing rule at g_j (that turns g_j to a constant). Since g_j is not the output gate, g_j is internal and therefore has fanout of at least 1 the moment it becomes constant. Thus, simplification removes at least one more costly gate above g_j in the topological order (Lemma 4) because of the constant fanout from g_j. Therefore, each iteration i eliminates at least 2 gates, and thus the for-loop eliminates at least $2|D_i| = 2N_i$ gates in this case.

After all variables in D_i are processed, the function computed by the restricted multiplexer still depends on a_i (because a_i determines whether the addressed data bit lies in D_i or in its complement among the remaining data bits). Since we assumed $\text{fanout}(a_i) \neq \emptyset$, the algorithm similarly picks a neighbor gate h_a of a_i, fixes a_i by fixer so that h_a becomes constant, and simplifies, eliminating at least one additional gate. Altogether, iteration i eliminates at least $2|D_i|+1 \;=\; N_i+1$ gates. Under the assumption that the algorithm does not terminate early, these eliminates occur for ever iteration $i = n, n-1, \ldots, 2$. The total number of gates eliminated after the loop terminates is then given by

$$\sum_{i=2}^{n}(N_i + 1) \;=\; \sum_{i=1}^{n}(2^i + 1) \;=\; 2 \cdot 2^n + n - 5 \;=\; 2N + \log N - 5$$

Since we assume $|C| < 2N + \log N - 3$, it must be the case that the size of the resulting circuit after the for-loop must be strictly less than 2 which violates the lower bound of MUX_1. Thus, the final brute-force step and find an explicit input on which C and MUX_1 differ. In either case, the algorithm produces an input (α, ρ) such that $C(\alpha, \rho) \neq \mathsf{MUX}_n(\alpha, \rho)$ as desired.

Finally, we argue that Algorithm 1 runs in polynomial time with respect to N. There are at most $\log N - 1$ loop iterations, and each condition inside the loop can be checked in polynomial time. Furthermore, applying substitution, simplification, and normalization on C in each iteration can be done in polynomial time since we only need to traverse the encoding of C to compute these steps. Finally, to extract a counterexample, we only need to compute the circuit on a constant number of inputs and each evaluation can be done in polynomial time. □

Algorithm 1 Refuter for MUX_n against circuits of size less than $2N + \log N - 3$.

Require: DeMorgan circuit C on address inputs $A = \{a_1, \ldots, a_n\}$ and data inputs $X = \{x_0, \ldots, x_{N-1}\}$ with $N = 2^n$ and $|C| < 2N + \log N - 3$

$\alpha \leftarrow \emptyset$; $\rho \leftarrow \emptyset$; $C_0 \leftarrow \textsc{Normalize}(C)$
▷ *working address restriction ; working data restriction ; working circuit* ◁
for $i \leftarrow n, n-1, \ldots, 2$ **do** ▷ *Process one address bit per iteration*
  $a \leftarrow a_i$ ▷ *Current last address bit*
  $D_i \leftarrow \{x_j \in X : \text{the } i\text{-th bit of } j-1 \text{ equals } 1\}$ ▷ *We assume the data bits are 1-indexed*
  for each $x_j \in D_i$ **do**
    if $\text{fanout}(x_j) = \emptyset$ **then**
      return "`DEGEN` in x_j under ρ" ▷ *C_i ignores x_j but* MUX *depends on it*
    $g_j \leftarrow$ topologically lowest gate in $\text{fanout}(x_j)$
    if **then**$(\neg)g_j$ is the output
      $\rho \leftarrow \rho \cup \{x_j \leftarrow c\}$
      return "`CONST` under ρ"
    $c \leftarrow \mathsf{fixer}(x_j, g_j)$
    $\rho \leftarrow \rho \cup \{x_j \leftarrow c\}$
    $C_i \leftarrow \textsc{Simplify}(C_i\restriction_{x_j \leftarrow c})$
  if $\text{fanout}(a) = \emptyset$ **then**
    return "`DEGEN` in a under α" ▷ *C_i ignores a_i but restricted* MUX *depends on it*
  $h_a \leftarrow$ topologically lowest gate in $\text{fanout}(a)$
  if **then**$(\neg)h_a$ is the output
    $\alpha \leftarrow \alpha \cup \{a \leftarrow d\}$
    return "`CONST` under α"
  $d \leftarrow \mathsf{fixer}(a, h_a)$
  $\alpha \leftarrow \alpha \cup \{a \leftarrow d\}$
  $C_i \leftarrow \textsc{Simplify}(C_i\restriction_{a \leftarrow d})$
  $X \leftarrow X \setminus D_i$ ▷ *Discard fixed data bits*
▷ *Now only a_1 remains: brute-force the remaining constant-size case* ◁
$\sigma \leftarrow$ brute-force over the constant number of remaining inputs and return the first assignment where $C(a_1, \sigma) \neq \mathsf{MUX}_1(a_1, \sigma)$
$\rho \leftarrow \rho \cup \sigma$
return "`FAIL` under α, ρ"

References

[AT11] Kazuyuki Amano and Jun Tarui. "A well-mixed function with circuit complexity 5n: Tightness of the Lachish–Raz-type bounds". In: *Theoretical computer science* 412.18 (2011), pp. 1646–1651.

[CDJ26a] Marco Carmosino, Ngu Dang, and Tim Jackman. *Convergent Gate Elimination and Constructive Circuit Lower Bounds*. 2026. arXiv: 2602.17942 [cs.CC]. URL: https://arxiv.org/abs/2602.17942.

[CDJ26b] Marco Carmosino, Ngu Dang, and Tim Jackman. "Simple Circuit Extensions for XOR in PTIME". In: *43rd International Symposium on Theoretical Aspects of Computer Science (STACS 2026)*. Ed. by Meena Mahajan, Florin Manea, Annabelle McIver, and Nguyen Kim Thang. Vol. 364. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2026, 23:1–23:20. ISBN: 978-3-95977-412-3. DOI: 10.4230/LIPIcs.STACS.2026.23. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.STACS.2026.23.

[CO25] Bruno Cavalar and Igor Oliveira. "Boolean Circuit Complexity and Two-Dimensional Cover Problems". In: *ACM Transactions on Computation Theory* 17.2 (2025), pp. 1–23.

[DK11] Evgeny Demenkov and Alexander S. Kulikov. "An Elementary Proof of a 3n - o(n) Lower Bound on the Circuit Complexity of Affine Dispersers". In: *Mathematical Foundations of Computer Science 2011 - 36th International Symposium, MFCS 2011, Warsaw, Poland, August 22-26, 2011. Proceedings*. Ed. by Filip Murlak and Piotr Sankowski. Vol. 6907. Lecture Notes in Computer Science. Springer, 2011, pp. 256–265. DOI: 10.1007/978-3-642-22993-0_25. URL: https://doi.org/10.1007/978-3-642-22993-0%5C_25.

[FGHK16] Magnus Gausdal Find, Alexander Golovnev, Edward A. Hirsch, and Alexander S. Kulikov. "A Better-Than-3n Lower Bound for the Circuit Complexity of an Explicit Function". In: *2016 IEEE 57th Annual Symposium on Foundations of Computer Science (FOCS)*. 2016, pp. 89–98. DOI: 10.1109/FOCS.2016.19.

[IM02] Kazuo Iwama and Hiroki Morizumi. "An Explicit Lower Bound of 5n - o(n) for Boolean Circuits". In: *Mathematical Foundations of Computer Science 2002, 27th International Symposium, MFCS 2002, Warsaw, Poland, August 26-30, 2002, Proceedings*. Ed. by Krzysztof Diks and Wojciech Rytter. Vol. 2420. Lecture Notes in Computer Science. Springer, 2002, pp. 353–364. DOI: 10.1007/3-540-45687-2_29. URL: https://doi.org/10.1007/3-540-45687-2%5C_29.

[Kar93] Mauricio Karchmer. "On proving lower bounds for circuit size". In: *[1993] Proceedings of the Eigth Annual Structure in Complexity Theory Conference*. IEEE. 1993, pp. 112–118.

[KP80] Klein and Paterson. "Asymptotically optimal circuit for a storage access function". In: *IEEE Transactions on Computers* 100.8 (1980), pp. 737–738.

[LR01] Oded Lachish and Ran Raz. "Explicit lower bound of 4.5n - o(n) for boolena circuits". In: *Proceedings of the Thirty-Third Annual ACM Symposium on Theory of Computing*. STOC '01. Hersonissos, Greece: Association for Computing Machinery, 2001, pp. 399–408. ISBN: 1581133499. DOI: 10.1145/380752.380832. URL: https://doi.org/10.1145/380752.380832.

[LY22] Jiatu Li and Tianqi Yang. "$3.1n$ - $o(n)$ circuit lower bounds for explicit functions". In: *STOC '22: 54th Annual ACM SIGACT Symposium on Theory of Computing, Rome, Italy, June 20 - 24, 2022*. Ed. by Stefano Leonardi and Anupam Gupta. ACM, 2022, pp. 1180–1193. DOI: 10.1145/3519935.3519976. URL: https://doi.org/10.1145/3519935.3519976.

[Pau75] Wolfgang J Paul. "A 2.5 n-lower bound on the combinational complexity of Boolean functions". In: *Proceedings of the seventh annual ACM symposium on Theory of computing*. 1975, pp. 27–36.

[Red73] NP Red'kin. "Proof of minimality of circuits consisting of functional elements". In: *Systems Theory Research: Problemy Kibernetiki* (1973), pp. 85–103.

[SC98] JE Savage and Models Of Computation. *Exploring the power of Computing*. 1998.

[Sch74] Claus-Peter Schnorr. "Zwei lineare untere Schranken für die Komplexität Boolescher Funktionen". In: *Computing* 13.2 (1974), pp. 155–171. DOI: 10.1007/BF02246615. URL: https://doi.org/10.1007/BF02246615.

[Sto77] Larry J. Stockmeyer. "On the Combinational Complexity of Certain Symmetric Boolean Functions". In: *Math. Syst. Theory* 10 (1977), pp. 323–336. DOI: 10.1007/BF01683282. URL: https://doi.org/10.1007/BF01683282.

[Weg87] Ingo Wegener. *The Complexity of Boolean Functions*. Wiley-Teubner, 1987.

[Wig93] Avi Wigderson. "The fusion method for lower bounds in circuit complexity". In: *Combinatorics, Paul Erdos is Eighty* 1.453-468 (1993), p. 68.

[Zwi91] Uri Zwick. "A 4n lower bound on the combinational complexity of certain symmetric boolean functions over the basis of unate dyadic Boolean functions". In: *SIAM Journal on Computing* 20.3 (1991), pp. 499–505.

A Recounting Paul's Lower Bound

In this section we will recount Paul's lower bound for the multiplexing function (MUX) [Pau75]. We note that Paul's basis was $\{\wedge, \oplus, \neg\}$ (with free $\neg$-gates and costly $\wedge, \oplus$-gates) which is different from our basis.

Still, the core ideas for the argument remain the same, an argument via Gate Elimination. Here, we simply provide our refined presentation of Paul's lower bound that aligns with our basis.

To this end, we define the set of i index selector functions as follows

Definition 7. Let $f : \{0,1\}^n \times \{0,1\}^{2^n}$ be a Boolean function with two distinguished sets of input variables: $a_1, \dots a_n$ the set of *address bits* and $x_1, \dots x_{2^n-1}$ be the set of *data bits*.

We define $A(f) = \{\alpha \in \{0,1\}^n : f\restriction_{a=\alpha} \equiv x_{(\alpha)}\}$ to be the set of *selectable addresses* of f.

We define S_i, the set of i index selector functions, as:

$$S_i = \{f : \{0,1\}^n \times \{0,1\}^{2^n} \to \{0,1\} : |A(f)| \geq i\}$$

In words, the set S_i contains Boolean functions f on $(n+2^n)$ input variables such that there are at least i addresses a satisfying $f(a,x) = x_{(a)}$. Thus, it is easy to see that when $i = 2^n$, we have $f \equiv \mathsf{MUX}_n$. With this observation, we obtain the following lower bound on MUX.

Theorem 8. *[Pau75] For the DeMorgan basis $\{\wedge, \vee, \neg\}$ where $\neg$-gates are for free, and for all $f \in S_i$, we have $CC(f) \geq 2i - 2$. Furthermore, $CC(\mathsf{MUX}_n) \geq 2N - 2$ where $N = 2^n$.*

Proof. Let C be an optimal circuit for some $f_i \in S_i$ (as in Definition 7) with $i \geq 1$, we proceed with proving the lower-bound by induction on i. The base case $i = 1$ is vacuous as $2 \cdot 1 - 2 = 0$ and the circuit complexity of all functions is non-negative.

For $i = 2$, we wish to show that $CC(f_2) \geq 2 \cdot 2 - 2 = 2$ for some $f_2 \in S_2$. By Definition 7, there are two selectable addresses α_1, α_2. By definition, f_2 is a Boolean function such that there exists at least two addresses $\alpha_1, \alpha_2 \in \{0,1\}^\ell$ such that $f_2(\alpha_1, x) = x_{(\alpha_1)}$ and $f_2(\alpha_2, x) = x_{(\alpha_2)}$. Since α_1 and α_2 corresponds to two different data bits, we need at least one $\wedge$-gate to select the appropriate appropriate data bit and one $\vee$-gate to combine the results of the selection (either one of them). Thus, $CC(f_2) \geq 2$.

For some $i = k \geq 1$, assume that $CC(f) \geq 2k - 2$ for any $f \in S_k$. Fix $f_{k+1} \in S_{k+1}$, we will show that $CC(f_{k+1}) \geq 2(k+1) - 2$. To this end, let C be an optimal circuit computing f_{k+1}. By definition of S_{k+1} we know $|A(f_{k+1})| \geq k+1$. Consider any $\alpha \in A$ and it's corresponding input $x_{(\alpha)}$. We analyze the following cases on the fan-out of $x_{(\alpha)}$ and argue the desired lower-bound for $|C|$ on all cases. First, we note that $\text{fanout}(x_{(\alpha)})$ cannot be 0. Assume towards contradiction that $\text{fanout}(x_{(\alpha)}) = 0$. Since f_{k+1} depends on $x_{(\alpha)}$ then $x_{(\alpha)}$ must be the output of the circuit (i.e. $f_{k+1} \equiv x_{(\alpha)}$ regardless of the value of a). Since $k \geq 1$, $|A(f_{k+1})| \geq 2$ and thus there exists an $a' \in A$ that is distinct from A. However by definition, $f_{k+1}\restriction_{a=\alpha'} \equiv x_{(\alpha')} \not\equiv x_{(\alpha)}$, a contradiction. We now analyze the remaining cases.

- Case 1: $\text{fanout}(x_{(\alpha)}) > 1$. In other words, $x_{(\alpha)}$ is feeds into at least two other costly gates in C. Fix $b \in \{0,1\}$ and consider C', the circuit obtained by moving any wires originating from $x_{(\alpha)}$ in C to the constant b and then performing standard gate elimination. We note that $|C'| \leq |C| - 2$ since $x_{(\alpha)}$ had at least two costly neighbors which would have been eliminated after our constant substitution.

  Let $f' : \{0,1\}^\ell \times \{0,1\}^N$ be the function C' computes. Observe that $A(f') = A(f) \setminus \{\alpha\}$. By the inductive hypothesis, $2^k - 2 \leq CC(f') \leq |C'| \leq |C| - 2 \leq CC(f_{k+1}) - 2$. Rearranging yields $CC(f_{k+1}) \geq 2k - 2 + 2 = 2(k+1) - 2$.

- Case 2: $\text{fanout}(x_{(\alpha)}) = 1$. This means that $(\neg)x_{(\alpha')}$ is fed into exactly one costly gate in C. Let us call this costly gate g. We first show that it is not the output of the circuit. If it were, notice that there is some constant we can substitute in for $x_{(\alpha')}$ that would eliminate g via a fixing rule and leaving the circuit constant. In other words, there exists a constant $b \in \{0,1\}$ such that $f_{k+1}\restriction_{x_{(\alpha)}=b}$ is degenerate with respect to all $x_{(\alpha')}$ for any $\alpha' \neq \alpha$ and $\alpha \neq \alpha'$. However, since $|A(f_{k+1})| \geq 2$, there is at least one other α'' such that $f_{k+1}\restriction_{a=\alpha'', x_{(\alpha)}=b} \equiv x_{(\alpha'')} \neq b$.

  Therefore, g is not the output gate and thus $(\neg)g$ feeds another costly gate $h \in \{\wedge, \vee\}$. Substituting the constant for $x_{(\alpha)}$ which eliminates g via a fixing rule will then also eliminate h as well. Thus, we obtain $CC(f_{k+1}) = |C| \geq 2(k+1) - 2$ via the same argument as in case 2.

Since $CC(f_{k+1}) \geq 2 \cdot 2^{k+1} - 2$ for both cases, the statement is true by the principle of mathematical induction. Set $i = 2^n = N$, we obtain $CC(\mathsf{MUX}_n) \geq 2N - 2$. $\square$

We will also show that Paul's lower bound is constructive as a supplemental result because the treatment for this refuter is also different compared to the refuter for our lower bound or the refuter of [CDJ26a] even though Schnorr's and Paul's technique are similar. The reason is because Paul's proof does not utilize the actual downward self-reducibility of MUX: a data bit restriction on a circuit computing MUX_n does not result in a circuit computing MUX_{n-1}.

Theorem 9. *Let C be a DeMorgan circuit on n inputs with size $s < 2(N-1)$, where $N = 2^n$, that claims to compute MUX_n. We can construct an input (α, ρ) with $\alpha \in \{0,1\}^n$, $\rho \in \{0,1\}^N$, and $N = 2^n$, such that $C(\alpha, \rho) \neq \mathsf{MUX}_n(\alpha, \rho)$ in polynomial time with respect to N.*

Proof. Let C be a DeMorgan circuit on address inputs $A = \{a_1, \ldots, a_n\}$ and data inputs $X = \{x_1, \ldots, x_{N-1}\}$, where $N = 2^n$, with $|C| < 2N - 2$. Algorithm 2 produces a tuple (α, ρ) (for the address and the data bits respectively) and either halts early in the `DEGEN`, `CONST` case or reaches the case `FAIL`, the final constant-size base case. We show that in all cases the algorithm yields a polynomial-time witness that $C \not\equiv \mathsf{MUX}_n$.

The algorithm maintains a sequence of simplified circuits $C_0, C_1, C_2 \ldots$ together with partial restrictions α on the address bits and ρ on the data bits. By construction (normalization at initialization, and simplification after each substitution), for every round i we have $C_i \equiv C\restriction_{\alpha,\rho}$, and C_i is kept in the chosen normal form.

Fix any round i, and let x_j be the selected data variable (with $j = (\gamma) + 1$ for the chosen address γ), recall that we define the data bits to be 1-indexed. We first argue that if the algorithm terminates early either at `DEGEN` or `CONST` case, we can construct a valid counterexample (α, ρ). To this end, we consider the following cases

1. If $\mathrm{fanout}((\neg)x_j) = 0$, then C_i does not depend on x_j. However, MUX_n depends on every data bit: under address γ, we have $\mathsf{MUX}_n(\gamma, x) = x_j$. Therefore there exist two completions of the current restriction that differ only on x_j but yield different values of MUX_n, while C_i (hence C under the same restriction) outputs the same value on both.

2. If the relevant costly neighbor (in either fanout case) is the output gate, then by fixing the corresponding literal the output of C_i becomes constant. But MUX_n cannot become constant after fixing a single data bit, since there exists another address selecting a different (unfixed) data bit. Thus, we can can explicitly extend the current restriction to obtain the desired (α, ρ) such that $C(\alpha, \rho) \neq \mathsf{MUX}_n(\alpha, \rho)$.

We now argue that if the algorithm does not terminate early, then it removes at least 2 costly gates from the circuit. To this end, we consider the following cases

1. When $\mathrm{fanout}_{C_i}((\neg)x_j) = 1$, let h be the (unique) costly gate reading $(\neg)x_j$. If h is the output gate, the algorithm returns a witness (handled in Step 2 below), so assume h is not the output. Let h' be the topologically-minimal costly successor of h. The algorithm sets x_j to $b = \mathsf{Fixer}(h, x_j)$. By definition of fixing rules, this assignment fixes the value of h, so h is eliminated after simplification. Since h is not the output, fixing h also fixes the input of h' along the unique path out of h, and in normal form this causes h' to be eliminated as well. Hence $|C_{i+1}| \leq |C_i| - 2$.

2. When $\mathrm{fanout}_{C_i}((\neg)x_j) \geq 2$, then there exist two distinct costly gates $h \neq f$ reading $(\neg)x_j$. If $(\neg)f$ is the output gate, the algorithm returns a witness, so assume f is not the output. Let f' be the topologically-minimal costly successor of f. The algorithm sets x_j to $b = \mathsf{Fixer}(f, x_j)$. As above, this fixes (and hence eliminates) f and then eliminates f' after simplification, yielding $|C_{i+1}| \leq |C_i| - 2$ (we, in fact, eliminate more than two gates in this case).

After some t iterations, we have $|C_t| \leq s - 2t$. Each successful round fixes exactly one data bit, so the residual task is still a multiplexer-type function on at least $N - t$ remaining selectable data positions. But since $s < 2(N-1)$, we have

$$|C_t| \leq s - 2t < 2(N-1) - 2t = 2(N-t) - 2,$$

There are at most $t = N - 2$ iterations before we reach the case $|V| = 2$ which corresponds to a circuit computing MUX_1. At this point the circuit has size $|C_t| < 2(N - N + 2) - 2 = 2$ which contradicts the lower bound of MUX_1 which requires at least 2 gates. Thus, we can brute-force the assignments on the remaining unfixed address bit and two data bits and complete α and ρ with one that yields the disagreement.

Finally, we argue that the procedure runs in polynomial time with respect to N. There are at most $N-1$ loop iterations, and each condition inside the loop can be checked in polynomial time. Furthermore, applying substitution, simplification, and normalization on C in each iteration can be done in polynomial time since we only need to traverse the encoding of C to compute these steps. Finally, to extract a counterexample, we only need to compute the circuit on a constant number of inputs and each evaluation can be done in polynomial time. □

Algorithm 2 Refuter for MUX_n against circuits of size $< 2(N-1)$.

Require: DeMorgan circuit C on address inputs $A = \{a_1, \ldots, a_n\}$ and data inputs $X = \{x_0, \ldots, x_{N-1}\}$ with $N = 2^n$ and $|C| < 2N - 2$

```
i ← 0; C_0 ← NORMALIZE(C)
α ← ∅                                   ▷ partial assignment to address bits
ρ ← ∅                                   ▷ partial assignment to data bits
A ← {0,1}^n                             ▷ addresses still consistent with α (implicit set)
V ← {0, ..., N−1}                       ▷ data indices not yet fixed by ρ
while |V| > 0 do
    Choose any address γ ∈ A and let j ← (γ) + 1 be its data index   ▷ (γ) is the base-10 number of γ
    if fanout((¬)x_j) = 0 then
        return "DEGEN in x_j under α, ρ"
    h ← topologically-minimal costly gate in C_i that reads (¬)x_j
    if fanout((¬)x_j) = 1 then          ▷ Case 2 in Paul's (single costly neighbor)
        if h is the output gate of C_i then
            return "CONST under α, ρ"
        else
            h' ← topologically-minimal costly successor of h
            b ← fixer(h, x_j)           ▷ b ∈ {0,1} that fixes h
            ρ ← ρ ∪ {x_j ↦ b}
            C_{i+1} ← SIMPLIFY(C_i ↾_{x_j=b})   ▷ eliminates h and h'
            i ← i + 1; V ← V \ {j}
    else                                ▷ Case 1 in Paul's (at least two costly neighbors)
        Let f be a second distinct costly gate (besides h) that reads (¬)x_j
        if (¬)f is the output gate of C_i then
            return "CONST under α, ρ"
        f' ← topologically-minimal costly successor of f
        b ← Fixer(f, x_j)
        ρ ← ρ ∪ {x_j ↦ b}
        C_{i+1} ← SIMPLIFY(C_i ↾_{x_j=b})   ▷ eliminates h, f, f' (at least 3 costly gates)
        i ← i + 1; V ← V \ {j}
▷ brute-force the remaining constant-size cases ◁
return "FAIL under α, ρ"
```

B Best Known Upper Bound Construction

To better understand the construction for MUX-circuits, we define a binary-to-positional decoder, an important component in the circuit construction, as follows

Definition 10 (Positional Decoder)**.** A function $\mathsf{DEC}_n : \{0,1\}^n \to \{0,1\}^{2^n}$ is called a binary-to-positional decoder if the following holds:

$$\mathsf{DEC}_n(a) = (sel_0(a), \ldots, sel_{2^n-1}(a))$$

where $sel_i(a) = 1$ iff $i = (\mathbf{a})$

Thus we can write the definition of MUX as a DNF using the output of DEC_n as follows

$$\mathsf{MUX}_n(a, x) = \bigvee_{i=0}^{2^n-1} sel_i(a) \wedge x_i$$

.

Furthermore, to simplify analysis and avoid irregularities arising in generic even or odd input lengths, we define and analyze restricted variants of these functions—Exponent Decoder (eDEC) and Exponent Multiplexer (eMUX) where the address length is always a power of two. This constraint enables recursive constructions that are clean, easily analyzable, and asymptotically optimal while remaining representative of the general case. In particular, we provided a fully formal and expanded version of the upper bound construction from [KP80] by first analyzing the construction for eDEC (Theorem 13) and then lift that to eMUX (Theorem 16). Namely,

Definition 11 (Exponent Positional Decoder and Multiplexer)**.** Let $\ell \in \mathbb{N}$, we define the classes of functions of Exponent Positional Decoders $\mathsf{eDEC}_\ell : \{0,1\}^{2^\ell} \to \{0,1\}^{2^{2^\ell}}$ and Exponent Multiplexers $\mathsf{eMUX}_\ell : \{0,1\}^{2^\ell} \times \{0,1\}^{2^{2^\ell}} \to \{0,1\}$ as follows

$$\mathsf{eDEC}_\ell(a) = (sel_0(a), \ldots, sel_{2^{2^\ell}-1}(a))$$

where $sel_i(a) = 1$ iff $i = (\mathbf{a})$

$$\mathsf{eMUX}_\ell(a, x) = \bigvee_{i=0}^{2^{2^\ell}-1} sel_i(a) \wedge x_i$$

Here, notice the subscript of ℓ used in the definition of Binary Positional Decoder and Multiplexer which no longer indicates the number of input variables. Instead, it serves as an "index" for the function in this particular family. So for example, eDEC_0 is the first and "smallest" one in the eDEC-function family and it corresponds to DEC_1 in the standard decoder family. Similarly, eDEC_1 is the second one in the family and corresponds to DEC_2, eDEC_2 to DEC_4, and so on. The same logic applies for the eMUX-function family.

B.1 Decoder Upper-Bound Construction

To begin with, we will look at a naive construction for a circuit computing DEC_n. Observe that for n address bits, there are $2^n = N$ possible minterms, and each minterm corresponds to exactly one output of DEC_n. Thus, our circuit construction is to simply compute all possible minterms composed by the address bits $a_1, \ldots, a_n$. To construct a minterm, we need $n - 1 = \log N - 1$ $\wedge$-gates. Therefore, this construction yields an upper bound $CC(\mathsf{DEC}_n) \leq (\log N - 1)N$ which is $O(N \log N)$ costly gates.

However, observe that we can achieve a better upper-bound by constructing the circuit for DEC_n recursively. Namely, we notice that by definition, minterms have a recursive property, i.e. a minterm on n variables is an AND of 2 smaller minterms, one is on half of the variables, and the other one is on the other half of the variables. That said, a better way to construct a DEC_n-circuit is to use two subcircuits, $\mathsf{DEC}_{\lfloor n/2 \rfloor}$-circuit and $\mathsf{DEC}_{\lceil n/2 \rceil}$-circuit, and combine every minterm generated by the first $\mathsf{DEC}_{\lfloor n/2 \rfloor}$-circuit with every minterm generated by the second $\mathsf{DEC}_{\lceil n/2 \rceil}$-circuit with $\wedge$-gates. We present Figure 2 regarding this construction in our upper bound proof in Section B.1 below. Thus, this construction yields a circuit complexity that is at most twice the complexity of $\mathsf{DEC}_{n/2}$-circuit plus N $\wedge$-gates that are used to combine the minterms of the two DEC-subcircuits.

Lemma 12 ([KP80; SC98])**.** *$CC(\mathsf{DEC}_n) \leq N + CC(\mathsf{DEC}_{\lfloor n/2 \rfloor}) + CC(\mathsf{DEC}_{\lceil n/2 \rceil}) = N + O(\sqrt{N})$, where $N = 2^n$*

Notice that each output entry of DEC_n corresponds to exactly one minterm the variables $\{a_1, a_2, \ldots, a_n\}$. Each minterm computes a distinct non-constant function which requires at least one costly gate to compute, and there are 2^n possible minterms. Thus, an obvious lower bound for the circuit complexity of DEC_n is $CC(\mathsf{DEC}_n) \geq N$ [SC98]. However, this lower bound only gives us information regarding the output layer

on the top of the circuit as n gets large which leaves rooms for improvement if one can witness a restriction that is guaranteed to eliminate gates in the intermediate levels of the circuit.

Below, we present a detailed analysis of the upper bound construction for eDEC. We note that our construction for the upper bound is based on that of [KP80; SC98], but we provide an exact count of gates for the case of eDEC.

Theorem 13. *Let $\ell \in \mathbb{N}$ and $N = 2^{2^\ell}$, then $CC(\mathsf{eDEC}_\ell) \leq N + \mathcal{S}(N)$ where $\mathcal{S}(N) = \sum_{i=1}^{\log\log N - 1} 2^i \cdot N^{1/2^i}$*

Proof. Let $r = s = \frac{2^\ell}{2} = 2^{\ell-1}$, then for some $a = b \circ c$, where $b \in \{0,1\}^r$, $c \in \{0,1\}^s$, and $i = 0, \ldots, 2^r - 1$, $j = 0, \ldots 2^s - 1$, we have

$$sel_{i \cdot 2^r + j}(a) = sel_{i \cdot 2^r + j}(b \circ c) = sel_i(b) \wedge sel_j(c)$$

which implies the following equation computing eDEC_ℓ defined in Definition 11

$$\mathsf{eDEC}_\ell(a) = \mathsf{eDEC}_\ell(b \circ c) = (sel_i(b) \wedge sel_0(c), \ldots, sel_i(b) \wedge sel_{2^j - 1}(c) \quad \text{for } i = 0, \ldots, 2^r - 1)$$

In words, for every $i = 0, \ldots, 2^s - 1$, we compute $sel_i(b) \wedge sel_j(c)$ for every $j = 0, \ldots, 2^r - 1$, so $sel_0(a)$ corresponds to $sel_i(b) \wedge sel_j(c)$ where $(i, j) = (0, 0)$, and $sel_1(a)$ corresponds to the term where $(i, j) = (0, 1)$, and so on.

We observe the following recursive construction for D_ℓ based on the equation above. In particular, let D_ℓ be the circuit computing eDEC_ℓ, then the construction of D_ℓ uses two $D_{\ell-1}$ sub-circuits and the output of D_ℓ are given by connecting the outputs of the two $D_{\ell-1}$ in the following manner: for each output of one $D_{\ell-1}$, we connect it to each output of the other $D_{\ell-1}$ using an $\wedge$-gate which requires a total of N $\wedge$-gates. Thus, we obtain

$$CC(\mathsf{eDEC}_\ell) \leq 2 \cdot CC(\mathsf{eDEC}_{\ell-1}) + N$$

We can repeat this process to construct $D_{\ell-1}$ using two sub-circuits $D_{\ell-2}$, and so on which suggests a recurrence for the construction of eDEC_ℓ-circuits. In particular, observe that the base case of the recursive construction is when $\ell = 0$ which is a simple decoder that has $2^{2^0} = 2$ outputs. This decoder requires zero costly gate since we have only two possible minterms, i.e. either the variable itself or its negation, so $CC(\mathsf{eDEC}_0) = 0$. Thus, we can express the upper-bound of $CC(\mathsf{eDEC}_\ell)$ via the following recurrence with $N = 2^{2^\ell} \implies \sqrt{N} = 2^{2^\ell/2} = 2^{2^{\ell-1}}$ (meaning decreasing ℓ by 1 accounts for taking the square root on the number of outputs)

$$T(N) = \begin{cases} 0, & \text{if } N = 2 \\ 2 \cdot T(N^{1/2}) + N, & \text{if } N > 2 \end{cases}$$

Now, we compute the number of steps to reach the base case. Let this number be k, then we have $1 = \frac{\ell}{2^k} \implies 2^k = \log N \implies k = \log\log N$. Next, we unroll the recurrence for k steps.

$$\begin{aligned}
T(N) &= 2 \cdot T(N^{1/2}) + N = 2(2 \cdot T(N^{1/4}) + N^{1/2}) + N \\
&= 4 \cdot T(N^{1/4}) + 2 \cdot N^{1/2} + N \\
&= 8 \cdot T(N^{1/8}) + 4 \cdot N^{1/4} + 2 \cdot N^{1/2} + N \\
&\ldots \\
&= 2^k \cdot T(2) + \sum_{i=0}^{k-1} 2^i \cdot N^{1/2^i} \\
&= \log N \cdot 0 + N + \sum_{i=1}^{\log\log N - 1} 2^i \cdot N^{1/2^i} \\
&= N + \sum_{i=1}^{\log\log N - 1} 2^i \cdot N^{1/2^i}
\end{aligned}$$

Setting $\mathcal{S}(N) = \sum_{i=1}^{\log\log N - 1} 2^i \cdot N^{1/2^i}$, we have $CC(\mathsf{eDEC}_\ell) \leq N + \mathcal{S}(N)$ as desired. □

Remark 14. *Our upper bound for* eDEC_ℓ *is consistent with the lower bound of* DEC_n *in Lemma 12. In particular, the first term in* $\mathcal{S}(N)$, $2 \cdot N^{1/2}$, *dominates the others, and thus, we believe one can show that* $\mathcal{S}(N) = O(\sqrt{N})$. *Furthermore, a useful thing to try is to derive the exact count for the general case of* DEC_n *where* n *is even or odd, each may differ by a few gates, but should be in* $O(\sqrt{N})$ *still.*

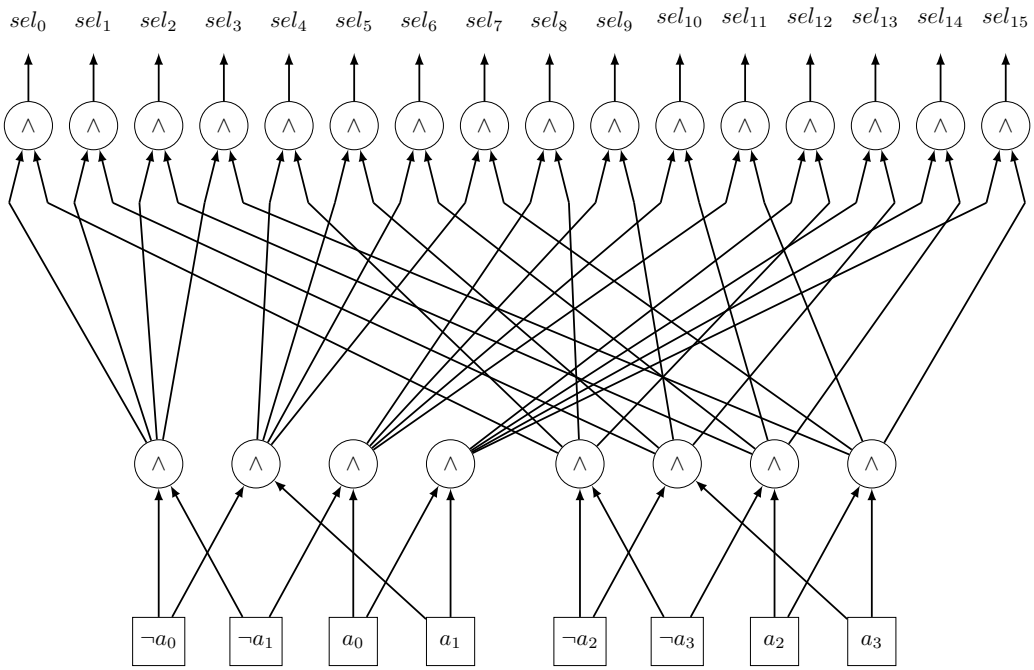

Figure 2: The figure shows the construction of DEC_4 that uses two DEC_2 subcircuits. The total complexity of this construction is $16 + 2 \cdot 16^{1/2^1} = 16 + 8 = 24$ costly gates. For readability, we simplify the negations of the input variables by treating each negation as its own input node.

B.2 Multiplexer Upper-Bound Construction

We begin with establishing an upper bound for MUX_n, and we start with the naive circuit construction. In particular, the construction follows from the alternative definition of MUX_n via Definition 10 of DEC_n. As a reminder,

$$\mathsf{MUX}_n(a, x) = \bigvee_{i=0}^{2^n-1} sel_i(a) \wedge x_i$$

In particular, it uses a total of $N - 1$ $\vee$-gates and N $\wedge$-gates, where $N = 2^n$ (see Figure 3). Thus, $CC(\mathsf{MUX}_n) \leq 2N - 1 + CC(\mathsf{DEC}_n) \leq 3N + O(\sqrt{N})$.

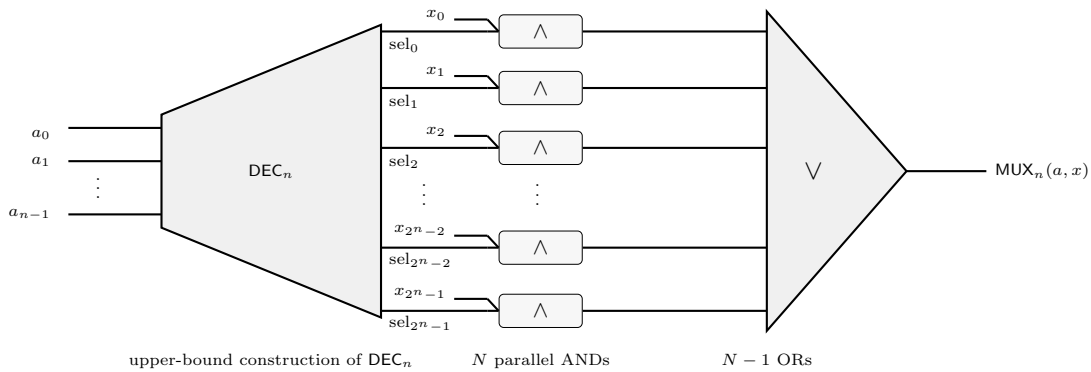

Figure 3: Naïve MUX_n-circuit construction (on input address bits $a_0, \ldots, a_{n-1}$ and data bits $x_0, \ldots, x_{2^n-1}$). The decoder DEC_n (on inputs $a_0, \ldots, a_{n-1}$) emits a one-hot vector $(sel_0, sel_1, \ldots, sel_{2^n-1})$. Each x_i and sel_i are fed into an AND-gate, and the results are combined by an OR-circuit to produce the final output.

The "overhead" of this construction is upper bounded by $2N-1$, but it turns out that we can also apply a recursive construction for MUX_n to obtain a better upper bound for the overhead. A better construction proposed by Klein and Paterson [KP80] takes advantage of the downward self-reducibility of MUX. In particular, the "overhead" of N costly gates can be reduced by using two smaller decoders *in sequence*, selecting part of the address with respect to different blocks of arguments in turn (see Figure 4). Namely, we have the following upper bound

Lemma 15 ([KP80])**.** *$CC(\mathsf{MUX}_n) \leq 2N + O(\sqrt{N})$, where $N = 2^n$*

Proof. Let $r = \lfloor \frac{n}{2} \rfloor$, $s = \lceil \frac{n}{2} \rceil$, and let D_r, D_s denote the circuits computing DEC_r, DEC_s respectively. We have the following sequence of equations that uses D_r and D_s in sequence.

$$\begin{aligned}
\mathsf{MUX}_n(a,x) = \mathsf{MUX}_n(b \circ c, x) &= \bigvee_{i=0}^{2^r-1} \bigvee_{j=0}^{2^s-1} (sel_i(b) \wedge sel_j(c) \wedge x_{i \cdot 2^r + j}) \\
&= \bigvee_{i=0}^{2^r-1} \left(sel_i(b) \wedge \left(\bigvee_{j=0}^{2^s-1} (sel_j(c) \wedge x_{i \cdot 2^r + j}) \right) \right) \\
&= \mathsf{MUX}_r \left(b, \left(\bigvee_{j=0}^{2^s-1} (sel_j(c) \wedge x_{i \cdot 2^r + j}) \text{ , for } i = 0, \cdots, 2^r - 1 \right) \right)
\end{aligned}$$

The circuit construction in this case is as follows (which is suggested by the middle equation): we first use D_s to select the appropriate j given by its binary presentation c. Then, for this "fixed" j, the term $sel_j(c) \wedge x_{i \cdot 2^r + j}$ can "filter out" the input indices that do not correspond to j. In other words, at this stage, we are asking the question: for all possible prefixes $b \in \{0,1\}^r$, what is the only correct suffix c? Thus, the output of this stage is the remaining $2^{\lfloor n/2 \rfloor}$ data variables x's whose suffix of the its index in binary is the string c. That said, we can ignore the inputs that do not meet this condition and proceed with selecting the other half b. Finally, we use D_r to select the other half b and the expression $b + 2^r \cdot c$ yields the correct index. Thus, we obtain a recursive computation of MUX_n in terms of MUX_r as suggested by the last equation.

We will now analyze the circuit complexity of this construction. In particular,

- the first stage of selecting the suffix c for the data bit's index in binary requires $2^r \cdot 2^s = 2^n$ $\wedge$-gates and $2^r(2^s - 1) = 2^n - 2^r$ $\vee$-gates. Specifically, there are 2^r possible prefixes $b \in \{0,1\}^r$, and for each b, we use 2^s $\wedge$-gates and $2^s - 1$ $\vee$-gates to select the correct c. Taking the complexity of D_s into account (i.e. $CC(\mathsf{DEC}_s)$), the total number of gates used in this stage is

$$CC(\mathsf{DEC}_s) + 2 \cdot 2^n - 2^r$$

- the second stage of selecting the prefix b for the data bit's index in binary, given a "fixed" suffix c requires 2^r $\wedge$-gates and $2^r - 1$ $\vee$-gates. Taking the complexity of D_r into account (i.e. $CC(\mathsf{DEC}_r)$), the total number of gates used in this stage is

$$CC(\mathsf{DEC}_r) + 2 \cdot 2^r - 1$$

Note that the construction of this stage is exactly like the naive approach but for MUX_r.

Therefore, by Lemma 12 and substituting $N = 2^n$ and $r = \lfloor n/2 \rfloor$, $s = \lceil n/2 \rceil$, the total complexity of this construction is upper bounded by

$$CC(\mathsf{MUX}_n) \leq CC(\mathsf{DEC}_r) + CC(\mathsf{DEC}_s) + 2 \cdot 2^n + 2^r - 1 = 2N + O(\sqrt{N})$$

□

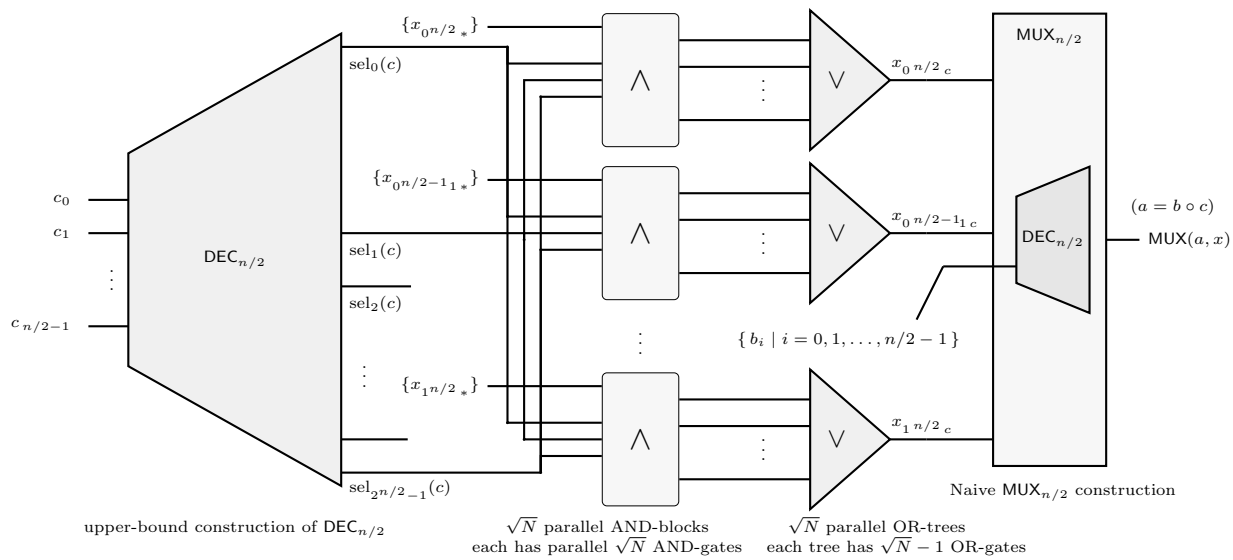

Figure 4: One-level expansion of MUX_n. The construction of $\mathsf{MUX}_{n/2}$-circuit on the right side follows the naive construction as in Figure 3.

The same analysis can be used to show an upper bound for eMUX_ℓ with an exact count rather than bigO. In particular, apply the upper bound in Theorem 13 for $\mathsf{eDEC}_{\ell-1}$, we obtain

$$\begin{aligned} CC(\mathsf{eMUX}_\ell) &\leq 2N + \sqrt{N} - 1 + 2 \cdot CC(\mathsf{eDEC}_{\ell-1}) \\ &= 2N + \sqrt{N} - 1 + 2(\sqrt{N} + \mathcal{S}(\sqrt{N})) \\ &= 2N + 3\sqrt{N} + 2 \cdot \mathcal{S}(\sqrt{N}) - 1, \quad \text{where } \mathcal{S}(\sqrt{N}) = \sum_{i=1}^{\log\log\sqrt{N}-1} 2^i \cdot (\sqrt{N})^{1/2^i} \end{aligned}$$

which is consistent with [KP80]. However, Klein and Patterson pointed out that the analysis of Lemma 15 suggests a recursive construction which can give further improvement to the $O(\sqrt{N})$ term, and in particular, decrease the coefficient of the term $\sqrt{N}$ for eMUX_ℓ.

The analysis in Lemma 15 suggests a recursive construction for the circuit which can yield an improvement for the $3\sqrt{N}$ term. In particular, we notice that we can split the string b into two halves and continue "filtering" the remaining variables and repeat the process. With this observation, we establish the following upper bound for eMUX_ℓ.

Theorem 16. *Let $\ell \in \mathbb{N}$, $CC(\mathsf{eMUX}_\ell) \leq 2N + \mathcal{S}(N) + 3$, where $N = 2^{2^\ell}$, and $\mathcal{S}(N) = \sum_{i=1}^{\log\log N - 1} 2^i \cdot N^{1/2^i}$*

Proof. Let $r = s = 2^\ell/2$. We apply one more iteration of the recursive construction and observe the improvement. In particular, we expand the expression for eMUX_ℓ similar to Lemma 15 further by splitting b into b_1 and c_1 where each has length $r/2 = 2^\ell/4$, then use two subcircuits computing $\mathsf{eDEC}_{\ell-2}$, one to fix c_1 and the other to process b_1 accordingly.

$$\begin{aligned}
\mathsf{eMUX}_\ell(a, x) &= \mathsf{eMUX}_\ell(b_1 \circ c_1 \circ c_0, x) \\
&= \bigvee_{i=0}^{2^{r/2}-1} \bigvee_{j=0}^{2^{r/2}-1} \bigvee_{k=0}^{2^s-1} (sel_i(b_1) \wedge sel_j(c_1) \wedge sel_k(c_0) \wedge x_{i\cdot 2^r + j\cdot 2^{r/2}+k}) \\
&= \bigvee_{i=0}^{2^{r/2}-1} \left(sel_i(b_1) \wedge \left(\bigvee_{j=0}^{2^{r/2}-1} \left(sel_j(c_1) \wedge \left(\bigvee_{k=0}^{2^s-1} (sel_k(c_0) \wedge x_{i\cdot 2^r + j\cdot 2^{r/2}+k}) \right) \right) \right) \right) \\
&= \mathsf{eMUX}_{\ell-2} \left(b_1, \left(\bigvee_{j=0}^{2^{r/2}-1} \left(sel_j(c_1) \wedge \left(\bigvee_{k=0}^{2^s-1} (sel_k(c_0) \wedge x_{i\cdot 2^r + j\cdot 2^{r/2}+k}) \right. \right. \right. \right. \\
&\qquad \left. \left. \left. \text{, for } i = 0, \ldots, 2^{r/2} - 1 \right) \right) \right) \right)
\end{aligned}$$

We will now analyze the circuit complexity of this construction. In particular,

- The first stage of fixing c_0 is the same as the analysis in Lemma 15 which requires

$$CC(\mathsf{eDEC}_{\ell-1}) + 2 \cdot 2^{2^\ell} - 2^{2^\ell/2}$$

- The second stage of selecting the c_1-part for the index requires $2^{\ell/2}$ $\wedge$-gates and $2^{r/2}(2^{r/2}-1)$ $\vee$-gates since this stage goes through all $2^{\ell/2}$ indices to select the c_1-part and combine. Taking $CC(\mathsf{eDEC}_{\ell-2})$ into account, the total number of gates used in this stage is

$$CC(\mathsf{eDEC}_{\ell-2}) + 2^{2^{\ell/2}} + 2^{2^{\ell/4}}(2^{2^{\ell/4}} - 1) = CC(\mathsf{eDEC}_{\ell-2}) + 2 \cdot 2^{2^{\ell/2}} - 2^{2^{\ell/4}}$$

- The third stage of selecting the b_1-part for the index, given fixed c, c_1-parts requires $2^{\ell/4}$ $\wedge$-gates and $2^{\ell/4} - 1$ $\vee$-gates. Taking the complexity $CC(\mathsf{eDEC}_{\ell-2})$, the total number of gates used in this stage is

$$CC(\mathsf{eDEC}_{\ell-2}) + 2 \cdot 2^{2^{\ell/4}} - 1$$

Substituting $N = 2^{2^\ell}$ and apply Theorem 13 for $CC(\mathsf{eDEC}_{\ell-1})$ and $CC(\mathsf{eDEC}_{\ell-2})$, we obtain

$$\begin{aligned}
CC(\mathsf{eMUX}_\ell) &\le CC(\mathsf{eDEC}_{\ell-1}) + 2 \cdot 2^{2^\ell} - 2^{2^{\ell/2}} + CC(\mathsf{eDEC}_{\ell-2}) + 2 \cdot 2^{2^{\ell/2}} - 2^{2^{\ell/4}} + CC(\mathsf{eDEC}_{\ell-2}) + 2 \cdot 2^{2^{\ell/4}} - 1 \\
&= 2 \cdot 2^{2^\ell} + 2^{2^{\ell/2}} + 2^{2^{\ell/4}} + CC(\mathsf{eDEC}_{\ell-1}) + 2 \cdot CC(\mathsf{eDEC}_{\ell-2}) - 1 \\
&\le 2N + N^{1/2} + N^{1/4} + N^{1/2} + \mathcal{S}(N^{1/2}) + 2N^{1/4} + 2 \cdot \mathcal{S}(N^{1/4}) - 1 \\
&= 2N + 2N^{1/2} + 3N^{1/4} + \mathcal{S}(N^{1/2}) + 2 \cdot \mathcal{S}(N^{1/4}) - 1
\end{aligned}$$

where $\mathcal{S}(M) = \sum_{i=1}^{\log\log M - 1} 2^i \cdot M^{1/2^i}$. We can repeat the process for $\ell = \log\log N$ times and express the upper bound of $CC(\mathsf{eMUX}_\ell)$ as the following recurrence

$$T(N) = \begin{cases} 3, & \text{if } N = 2 \\ T(N^{1/2}) + 2N + \mathcal{S}(N^{1/2}), & \text{if } N > 2 \end{cases}$$

For the base case $N = 2$ (i.e. $\ell = 0$), we have eMUX_0 which is equivalent to multiplexing 2 data bits x_0, x_1. We can construct the circuit by using a eDEC_0-circuit which requires zero costly gate because it is simply the input variable itself (a_0) or its negation ($\overline{a_0}$). Then, we construct $\overline{a_0} \wedge x_0$, $a_0 \wedge x_1$ and combine their results with an $\vee$-gate which yields a total of 3 costly gates.

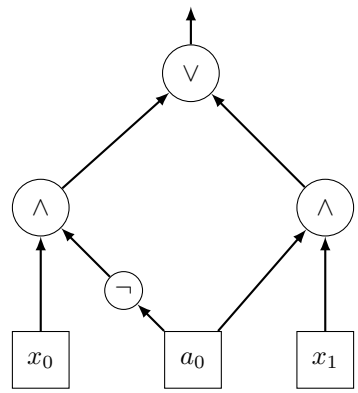

Figure 5: A circuit computing eMUX_0

Justification for the case of $N > 2$ is based on the analysis of performing one recursive step as in Lemma 15 as shown below

$$\begin{aligned} CC(\mathsf{eMUX}_\ell) &\leq \underbrace{CC(\mathsf{eDEC}_{\ell-1}) + (2N - \sqrt{N})}_{\text{selecting data bits based on an address slice stage}} + CC(\mathsf{eMUX}_{\ell-1}) \\ &\leq \underbrace{\sqrt{N} + \mathcal{S}(\sqrt{N})}_{\text{apply Theorem 13 for } CC(\mathsf{eDEC}_{\ell-1})} + (2N - \sqrt{N}) + \underbrace{CC(\mathsf{eMUX}_{\ell-1})}_{T(N^{1/2})} \\ &= 2N + \mathcal{S}(N^{1/2}) + T(N^{1/2}) = T(N) \end{aligned}$$

Now we unroll the recurrence for $\log\log N$ steps (until the data bits domain shrink to the base case)

$$\begin{aligned} T(N) &= T(N^{1/2}) + 2N + \mathcal{S}(N^{1/2}) \\ &= T(N^{1/4}) + 2N^{1/2} + \mathcal{S}(N^{1/4}) + 2N + \mathcal{S}(N^{1/2}) \\ &= T(N^{1/8}) + 2N^{1/4} + \mathcal{S}(N^{1/8}) + 2N^{1/2} + \mathcal{S}(N^{1/4}) + 2N + \mathcal{S}(N^{1/2}) \\ &\dots \\ &= T(2) + \sum_{i=0}^{\log\log N - 1} (2N^{1/2^i} + \mathcal{S}(N^{1/2^{i+1}})) \\ &= 3 + \sum_{i=0}^{\log\log N - 1} (2N^{1/2^i} + \mathcal{S}(N^{1/2^{i+1}})) \end{aligned}$$

Note that we can simplify the expression further by simplifying $\sum_{i=0}^{\log\log N-1}(2N^{1/2^i} + \mathcal{S}(N^{1/2^{i+1}}))$. In particular, for each i, we expand $\mathcal{S}(N^{1/2^{i+1}}) = \sum_{j=1}^{\log\log N - i - 2} 2^j N^{1/2^{i+1+j}}$. For readability, set $L = \log\log N$, then, we have

$$\sum_{i=0}^{L-1}(2N^{1/2^i} + \mathcal{S}(N^{1/2^{i+1}})) = \sum_{i=0}^{L-1} 2N^{1/2^i} + \sum_{i=0}^{L-1}\sum_{j=1}^{L-i-2} 2^j N^{1/2^{i+1+j}}$$

The index of the inner sum runs up to $L - 1 - 2$ because by definition $\mathcal{S}(M) = \sum_{i=1}^{\log\log M - 1} 2^i \cdot M^{1/2^i}$, then for $M = N^{1/2^{i+1}}$, we have $\log\log M - 1 = L - (i+1) - 1 = L - i - 2$.

Re-index the double sum with $u = i + j + 1$, then since $j \geq 1$, we have $u - 1 - i \geq 1 \iff i \leq u - 2$. Thus,

$$\sum_{i=0}^{L-1}\sum_{j=1}^{L-i-2} 2^j N^{1/2^{i+1+j}} = \sum_{u=1}^{L-1}\sum_{i=0}^{u-2} 2^{u-i-1} N^{1/2^u}$$

Rename the index of the single sum with u and factor out the first term of the sum, we have

$$\sum_{i=0}^{L-1} 2N^{1/2^i} = 2N + \sum_{u=1}^{L-1} 2N^{1/2^u}$$

Putting everything together, we have

$$\begin{aligned}\sum_{i=0}^{L-1} 2N^{1/2^i} + \sum_{i=1}^{L-1}\sum_{i=1}^{L-i-2} 2^j N^{1/2^{i+1+j}} &= 2N + \sum_{u=1}^{L-1} 2N^{1/2^u} + \sum_{u=1}^{L-1}\sum_{i=0}^{u-2} 2^{u-i-1} N^{1/2^u} \\ &= 2N + \sum_{u=1}^{L-1}\left(2 + \sum_{i=0}^{u-2} 2^{u-i-1}\right) N^{1/2^u}\end{aligned}$$

Finally, we evaluate $2 + \sum_{i=0}^{u-2} 2^{u-i-1}$ to determine the coefficient of $N^{1/2^u}$. To this end, re-index the sum with $k = u - i - 1$, and since $i = 0, 1, \ldots, u-2$, we have $k = u-1, u-2, \ldots, 1$. In other words,

$$2 + \sum_{i=0}^{u-2} 2^{u-i-1} = 2 + \sum_{k=1}^{u-1} 2^k = 2 + \frac{2(2^{u-1}-1)}{2-1} = 2^u \quad \text{(apply geometric series formula)}$$

Thus, substitute $L = \log\log N$ and $\mathcal{S}(N) = \sum_{i=1}^{\log\log N - 1} 2^i \cdot N^{1/2^i}$, we obtain the desired upper bound

$$T(N) = 3 + 2N + \sum_{u=1}^{\log\log N - 1} 2^u N^{1/2^u} = 2N + \mathcal{S}(N) + 3$$

□

If we unroll the first few terms in the sum $\mathcal{S}(N)$, we get $2N + 2\sqrt{N} + O(\sqrt[4]{N})$ which is a slight improvement over Lemma 15 which is $2N + 3\sqrt{N} + O(\sqrt[4]{N})$

Remark 17. *We think it can be beneficial for gaining a better understanding on how* MUX_n *works for general cases where* n *is either even or odd by getting an exact count of the upper bound for those cases. We think each may differ by a few gates, but the general form of the upper bound should still be* $2N + 2\sqrt{N} + O(\sqrt[4]{N})$.

B.3 Future Directions

An interesting research direction is to show that the bounds of MUX are tight and that the recursive construction of MUX as in Theorem 16 is optimal. This will be beneficial to show that MUX is a potential candidate for proving ETH-hardness of MCSP. The work of [CDJ26b] provides a characterization of the optimal circuit structures computing XOR_n which is a "tree" of $(n-1)$ XOR_2-sub-circuits via Gate Elimination alone. The key idea is to argue a particular shape at the bottom level of the circuit via an intricate case analysis to show that any deviation from the target shape will violate the tight bound of XOR. The XOR-function is well-behaved enough that we can feasibly exhaust all the cases. But, this approach is a major challenge for MUX because we will not only need to tight the bounds of MUX but also might have to argue that the decoder DEC is a "must have" which remains elusive to us using standard argument via Gate Elimination alone like what we have done throughout this thesis. In other words, we might have to come up with a much different argument via Gate Elimination by rigorously strengthening the usage of Lemma 4 or even the lemma statement itself with complicated combinatorics and counting to account for the local structure of the circuit around the address bits, the one thing that our proof of Theorem 8 is missing.

While such improvement remains ilusive to us, the observation motivates us to look at other circuit lower bound techniques to gain more insight on MUX, one of which is the *Fusion Method* which yields a lower bound on the AND-complexity of circuits in Negation Normal Forms (NNF) [Kar93; Wig93; CO25]. In the NNF regime, one can compute the OR-complexity of f which equals to the OR-complexity of $\neg f$. Even though we consider generic circuits throughout this work, we note that the provided upper bound constructions for DEC and MUX are already in NNF, and gaining some knowledge about MUX in the NNF

world is interesting and significant enough towards understanding the behavior of this function. In particular, knowing the lower bound on the "budget" of AND and OR-gates can be immensely useful in not only achieving a better lower bound but also characterizing the optimal circuit structure for MUX. Recently, Calavar and Oliveira recast Fusion Method using a modern set-theoretic language that works for various settings including non-monotone Boolean circuits in NNF [CO25]. They mention that reproving known circuit lower bounds via Gate Elimination, or better yet improving these lower bounds, for non-monotone functions using their framework remains an open problem.

Some promising research directions. Even though Fusion Method is only known to applied for DeMorgan circuits in NNF, rather than generic DeMorgan circuits which is the model of our work, it is reasonable to think that this approach is helpful for gaining more knowledge on the lower bound of DEC and MUX for various reasons.

- Under the NNF restriction, we can observe that the construction shows in Figure 2 is the only option for the upper-bound construction for DEC in the NNF world since we cannot have negations in the middle of the circuit. This construction shows a layer of AND-gates only which is compatible with how Fusion Method is useful for determining the AND-complexity (i.e. the lower bound on the number of AND-gates). Thus, if we can extend the framework in [CO25] for multi-output functions, then it is reasonable to believe that DEC-circuits in NNF must use *many* AND-gates and thus a tighter bound.

- Along this line, we observe that the full recursive upper bound construction for MUX-circuits as proposed in Theorem 16 is already in NNF assuming the NNF construction of DEC-circuits. Looking further in the construction, we observe that it uses noticeably more AND-gates compared to OR-gates when we shrink the overhead to just MUX_1 in the full recursive construction. Thus, it is also reasonable to believe that Fusion Method can help us show that the AND-complexity of MUX in NNF is high. By analyzing and knowing the exact AND-complexity of MUX, we believe one can gain more knowledge on how these AND-gates must be used structurally so that it does not violate the bounds.

- It is also interesting to see if one can extend Fustion Method for generic circuit models, or better yet, develop a sort of "hybrid" technique that combines Fusion Method and Gate Elimination arguments to prove better lower bounds.